

Roll No :- 263333251

ASSIGNMENT - 1

OPERATING SYSTEMS

* Types of Operating Systems:

1. Batch Operating System

Definition :- A batch operating system execute multiple jobs in batches without direct user interaction.

Usage :-

- ① Payroll processing
- ② Bank transaction processing
- ③ Large-scale data processing

Advantages :-

- ① Efficient for repetitive tasks
- ② Reduces idle CPU time
- ③ Can process large amounts of data

Disadvantages :-

- ① No direct user interaction
- ② Long waiting time for results
- ③ Difficult to debug errors

2. Time-sharing Operating System

Definition :- A Time-sharing operating system allows multiple users to use the computer simultaneously by sharing CPU time.

Usage :- ① University computer labs
② Multi-user servers
③ Office computer systems

Advantages :-

- ① Supports multiple users
- ② Fast response time
- ③ Better CPU utilization

Disadvantages :-

- ① Security issues
- ② Higher system complexity
- ③ Performance decreases with many users.

3. Multiprogramming Operating System

Definition :- A multiprogramming Operating System keeps multiple programs in memory and executes them efficiently to maximize CPU utilization.

Usage :- ① Data centers
② Mainframe computers
③ Business applications

Advantages :-

- ① Improves CPU utilization
- ② Increases system efficiency
- ③ Reduces CPU idle time

Disadvantages :-

- ① complex memory management
- ② Requires more memory

4. Multitasking Operating System

Definition :- A Multitasking Operating System allows a user to run multiple application at the same time.

Usage :- ① Personal computers

② Smartphones

③ Laptops

Advantages :-

① Increases productivity

② Easy switching between tasks

③ Efficient use of CPU

Disadvantages :-

① Requires more RAM

② Can slow down with many applications

③ More complex resource management

5. Multiprocessing Operating System

Definition :- A multiprocessing operating system uses two or more processors to execute multiple processes simultaneously.

Usage :- ① Scientific computing

② Engineering applications

③ High-performance servers

Advantages :-

① Faster processing

② Higher reliability

Disadvantages :-

- ① Expensive hardware
- ② complex design
- ③ Difficult synchronization

6. Real-Time operating System

Definition :- A Real-Time operating System provides immediate responses to events within a fixed time limit.

Usage :-

- ① Medical equipment
- ② Air traffic control
- ③ Robotics

Advantages :-

- ① Very fast response
- ② High reliability
- ③ Predictable performance

Disadvantages :-

- ① Expensive to develop
- ② Limited multitasking
- ③ Complex programming

7. Distributed operating system

Definition :- A distributed operating System manages multiple computers connected through a network as a single system.

Usage :- ① cloud computing
 ② Distributed databases
 ③ Large organizations

Advantages :-

- ① Resource sharing
- ② High reliability
- ③ Faster data processing

Disadvantages :-

- ① complex management
- ② Network dependency
- ③ Security challenges

8. Network Operating System

Definition :- A Network operating System manages network resources and allows computers to communicate over a network.

Usage :- ① office networks
 ② File Servers
 ③ web Servers

Advantages :-

- ① Easy resource sharing
- ② centralized management
- ③ Better Security

Disadvantages :-

- ① server dependency
- ② Higher setup cost
- ③ Requires network administration.

9. Embedded operating System

Definition:- An Embedded operating System is designed for dedicated devices with limited resources.

Usage:-

- ① ATMs
- ② Smart TVs
- ③ Cars

Advantages:-

- ① Small Size
- ② Fast performance
- ③ Low power consumption

Disadvantages:-

- ① Limited functionality
- ② Difficult to upgrade
- ③ Hardware - Specific

10. Mobile operating System

Definition:- A mobile operating System is designed specifically for smartphones & tablets.

Usage:-

- ① Smartphones
- ② Tablets
- ③ Smartwatches

Advantages:-

- ① User - friendly interface
- ② Supports mobile applications
- ③ Optimized battery usage

Disadvantages :-

- ① Limited compared to OS
- ② Security risk
- ③ Hardware limitations

* Types of System Calls

1. Process Control System Calls
- Used to create, execute, terminate, and manage processes
2. File Management System Calls
- Used to create, open, read, write, modify and delete files
3. Device Management System Calls
- Used to communicate with and control hardware devices
4. Memory Management System Calls
- Used to allocate, deallocate & manage memory
5. Information Maintenance System Calls
- Used to get or set system & process information

6. Communication System calls

- Used for communication between processes (IPC), such as pipes, sockets, etc.

7. Protection & Security System calls

- Used to control access permissions & protect system resources.

* System Calls

1. `fork()` - Creates a new child process by duplicating the calling process

2. `vfork()` - Creates a child process that shares the parent's address space until it calls `exec()` or `exit()`

3. `clone()` - Creates a new process or thread with customizable resource sharing.

4. `execve()` - Replaces the current process image with a new program.

5. `exit()` - Terminates a process after performing cleanup

6. `_exit()` - immediately terminates a process without standard cleanup.

7. `wait()` - waits for any child process to terminate
8. `waitpid()` - Waits for a specific child process to terminate
9. `waitid()` - Waits for process state changes and provides detailed status information
10. `kill()` - Sends a signal to a process or process group
11. `getpid()` - Returns the process ID (PID) of the calling process
12. `getppid()` - Returns the Parent Process ID
13. `nice()` - changes the scheduling priority of a process.
14. `setpriority()` - sets the scheduling priority of a process, process group or user
15. `getpriority()` - Retrieves the scheduling priority of a process, process group or user.
16. `open()` - opens an existing file & returns a file descriptor.

17. `creat()` - Creates a new file or truncate an existing one and opens it for writing.
18. `close()` - closes an open file descriptor
19. `read()` - Reads data from a file descriptor into memory
20. `write()` - Writes data from memory to a file descriptor
21. `pread()` - Reads data from a specified file offset without changing the current file position.
22. `pwrite()` - Writes data to a specified file offset without changing the position
23. `lseek()` - changes the current file offset of a file descriptor
24. `stat()` - Retrieves information about a file using its pathname.
25. `fstat()` - Retrieves information about a file using its file descriptor.
26. `mkdir()` - Creates a new directory

27. `lstat()` - Retrieves information about a symbolic link instead of the target file.
28. `access()` - checks a user's permissions for a file.
29. `unlink()` - Deletes a file by removing its directory entry
30. `rename()` - Renames or moves a file
31. `rmdir()` - Removes an empty directory
32. `dup()` - Duplicates an existing file descriptor
33. `dup2()` - Duplicates a file descriptor to a specified descriptor number.
34. `fcntl` - Performs various operations on file descriptors.
35. `fsync()` - flushes all modified file data
36. `fdatasync()` - flushes modified file data
37. `truncate()` - changes the size of a file
38. `ftruncate()` - changes the specified file.